# TAXOSHOP: Taxonomy-Guided Variant Analysis Across Programming Languages

***Abstract*—Interpreted languages such as Python, Ruby, and PHP are fundamental to modern software development, from powering servers to machine learning applications. CPython's repository in particular contains years of bug reports and related patches; however, traditional bug variant analysis techniques examine bugs individually, limiting their ability to generalize vulnerable patterns. We present TAXOSHOP, an approach that uses large language models to mine a project's bug history into a taxonomy of recurring bug patterns. TAXOSHOP abstracts bug reports into implementation-agnostic workflows that are clustered by similarity. For each cluster, TAXOSHOP generates a taxonomy entry that describes the shared bug patterns as a causal chain. Applied to over 300 CPython bug reports, TAXOSHOP identifies 5 primary systemic fault patterns. To demonstrate the taxonomy's utility, we use it to guide a simple LLM-based variant analyzer. The analyzer discovers 23 previously unknown bugs: 10 in CPython, 8 in MRuby, and 5 in the official PHP implementation, all confirmed by maintainers. Although mined solely from CPython, the taxonomy transfers across interpreters, and enables superior performance to unguided, CWE-guided, and instance-guided baselines.**

## I. INTRODUCTION

Interpreted languages like Python, Ruby, and PHP are essential to modern software. They power web backends, data science pipelines, machine learning infrastructure, and a large share of the world's websites [1]. Python in particular has become one of the most widely used programming languages [2], and its reference implementation, CPython, is used by millions of applications and packages.

Their widespread usage makes the correctness and security of their implementations important. The dominant implementations of these languages (CPython, CRuby, and the Zend Engine behind PHP) are large, memory-unsafe C codebases that expose rich object models to user code, and interleave interpreter-internal state with arbitrary user-defined behavior. Defects in the interpreter, such as a use-after-free triggered by a user-defined comparison method or an integer overflow in a built-in routine, affects every downstream application. Because interpreters are routinely used to execute untrusted or semi-trusted code (e.g., in sandboxes, notebooks, plugin systems, and multi-tenant services), such defects can escalate from crashes to exploitable vulnerabilities [3].

Simultaneously, these projects have accumulated a rich history failures. CPython's GitHub repository alone has approximately 27,000 bug reports spanning more than 25 years. Each report describing a triggering input, the faulty behavior, and often a root-cause analysis with fix. Past bug reports encode recurrent bug patterns, and those patterns can guide the discovery of new bugs.

Variant analysis seeks to leverage this insight. Traditional approaches rely on security experts to inspect a vulnerability, identify its characteristics, and manually audit code or construct analysis rules [4]. Automated systems reduce this burden by deriving search signatures, similarity representations, or static-analysis queries from a vulnerable program and its patch [5], [6], [7], [8], [9], [10].

Existing variant analysis techniques primarily extrapolate from *individual* bugs, confining variant analysis. Two bugs can arise from the same underlying programming mistake while sharing little source-level structure, occurring in unrelated subsystems or involving different APIs. Thus, searching for variants of individual bug instances can miss bugs that share its failure mechanism but not its surface form. Even approaches that explicitly derive higher-level variant patterns construct those patterns from individual vulnerabilities rather than discovering high-level, recurring mechanisms across a historical bug corpus.

**Insight.** Effective variant analysis using a large bug report corpus requires a generalized taxonomy which characterizes and clusters bug report abstractions. Abstracting away instance-specifics such as functions, variables, and files enables the creation of abstract bug workflows which describe the high-level characteristics of individual failures. Furthermore, effective abstraction enables taxonomy usage across language interpreters.

**Approach.** We present TAXOSHOP, a system for mining historical interpreter bugs into reusable bug-pattern taxonomies and applying those taxonomies to variant analysis. Starting from a corpus of CPython bug reports and their corresponding fixes, TAXOSHOP first constructs a normalized analysis of each bug and abstracts it into a bug workflow that captures its causal structure while removing implementation-specific details. It then groups related workflows and analyzes each group to derive a generalized description of a recurring bug mechanism supported by multiple historical instances.

**Results.** Our cluster analysis of 361 mined CPython bug reports identifies 10 primary bug taxonomies. Using these taxonomies to guide our agentic system, we discover 23 previously unknown bugs across three interpreters: 10 in CPython, 8 in Ruby, and 5 in PHP. All discovered bugs have been confirmed and 16 have been fixed by the respective maintainers. These results show that taxonomies derived from one interpreter's bug history transfer to other interpreters, enabling cross-project variant analysis.

In summary, this paper makes the following contributions:

- **A mining approach** that uses LLMs to abstract interpreter bug reports into high-level bug workflow descriptions and clusters them into bug taxonomies.
- **An empirical taxonomy** of CPython bugs, comprising 10 primary categories derived from 361 bug reports, along with an analysis of their characteristics.
- **A taxonomy-guided agentic system** that uses these categories to direct the search for new bugs in CPython and other C-based interpreters.
- **An evaluation** demonstrating the discovery of 20 new bugs across CPython, Ruby, and PHP, showing that bug patterns transfer across interpreter implementations.
- **An open-source artifact** containing our mined dataset, taxonomies, and tooling [**?**].

## II. Background

**Scripting Languages.** The reference implementations of Python, Ruby, and PHP (CPython, CRuby, and the Zend Engine) are all large C programs. Each implements a dynamic object model in native code: built-in types, the standard library's core modules, and extensions that wrap external C libraries are written in C and manipulate interpreter objects directly. These interpreters share two properties that make memory-safety bugs both common and structurally similar.

First, native code routinely calls back into user code. Operations that appear primitive at the language level, such as comparing two values, converting an argument to an integer, or iterating over a container, are dispatched to methods that users can override. Examples include Python's `__eq__`, `__hash__`, and `__index__`; Ruby's implicit conversion methods `to_ary`, `to_int`, and `to_str`; and PHP's magic methods and user-supplied callbacks. A native routine that invokes such a method hands control to arbitrary code. That code can mutate, resize, or free any object the routine is still using.

Second, the interpreters manage memory manually beneath a garbage-collected surface. CPython and PHP use reference counting supplemented by a cycle collector. CRuby uses a generational, incremental mark-and-sweep collector that scans the native stack conservatively. In all three, native code must track which references it owns. CPython makes this explicit with its distinction between *owned* and *borrowed* references: a borrowed reference is valid only as long as some other owner keeps the object alive. A borrowed reference, raw pointer, or cached length that outlives its owner becomes a dangling handle.

Concurrency adds a further hazard. CPython and CRuby serialize most execution under a global lock, but not all shared state is protected by it. CPython now also offers a free-threaded build without a global lock [**?**], which exposes shared state that the lock previously protected implicitly.

**Variant Analysis.** Variant analysis is the practice of finding new bugs that share a root cause with a known one [**?**]. It is widely used in security practice because fixed bugs are often not isolated: the flawed assumption behind one bug frequently recurs elsewhere in the same codebase, and a fix for one instance rarely corrects the others. In its most common form, variant analysis is manual. An expert studies a fixed bug, generalizes its root cause, and either audits related code or writes a query that encodes the pattern in a static-analysis framework such as CodeQL [**?**] or Semgrep [**?**]. The quality of the result depends on the expert's generalization, and each query is typically written for, and tested against, a single bug.

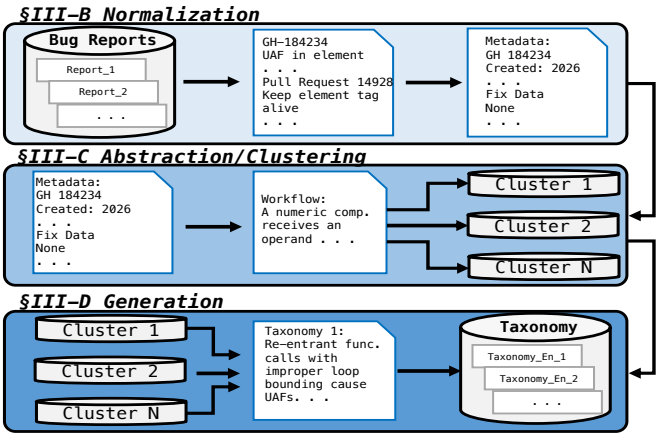



**Fig. 1: Overview of TaxoShop**

Automated approaches generally match code against individual known vulnerabilities. Vulnerable code clone detectors such as ReDeBug [**?**] and VUDDY [7] locate code that is syntactically similar to a known vulnerable function. Patch-based techniques such as MVP [**?**] derive signatures from a vulnerability and its fix to find unpatched instances. Directed fuzzers such as AFLGo [**?**] concentrate testing effort near locations associated with known bugs. More recently, LLM-based agents have been applied to variant analysis, starting from a single recently fixed bug and searching for similar code [**?**].

These approaches share two limitations. First, they operate at the granularity of a single bug, so they find variants that resemble that bug's surface form but miss bugs that share its root cause through different code. Second, their signatures, queries, and seeds are tied to one codebase's identifiers and idioms, so knowledge gained on one project does not carry over to another. TaxoShop addresses both limitations by abstracting over many bugs at once and away from the project in which they were found.

## III. TaxoShop

This sections presents the detailed design of TaxoShop. We first summarize the three core challenges in generating a taxonomy for variant analysis then describe the three stages of TaxoShop along with its paired variant analyzer. Figure 1 illustrates the overview of TaxoShop.

### A. Challenges

Automatically generating a practical taxonomy which can guide variant analysis introduces three primary challenges, each of which motivates the design of TaxoShop.

**C1: Abstraction Level.** The major shortcoming of current variant analysis techniques is fine-grained bug extrapolation. Because these techniques emphasize instance-based variants, they uncover bugs near the original instance but neglect the discovery of the same bug pattern in a different subsystem or adjacent application. Conversely, general-purpose weakness taxonomies such as the Common Weakness Enumeration (CWE) over-generalize. For example, simply characterizing a bug as a use-after-free (CWE-416) does not provide actionable information on how to discover similar errors.

**C2: Pattern Inference.** To create an accurate taxonomy, recurrent bug patterns must be inferred. Determining underlying patterns requires clustering similar bugs before examining clusters from a macro-perspective. However, reports of the same mechanism often rarely share vocabulary, so surface-level similarity groups bugs by module instead of root cause. Furthermore, a single report cannot reveal which of its details are essential to the pattern and which are incidental; this can only be determined by comparing every report in a cluster.

**C3: Taxonomy Structure.** A taxonomy must formally characterize bug clusters. Characterizing a cluster by its symptoms is insufficient, as the same root cause can manifest as different symptoms (e.g., a stale reference may cause either a use-after-free or an out-of-bounds read). Each entry must instead capture the causal chain leading to the failure, and it must be actionable: it should state where to search for new instances and where the pattern stops applying.

### *B. Report Normalization*

Practically, repository bug reports are heterogeneous in structure and level of detail. TAXOSHOP therefore normalizes each report into a uniform, agent-readable document. This stage is deterministic and purely structural. TAXOSHOP collects the information associated with a report and renders it into a fixed layout without interpreting the underlying bug. Critical evidence, including sanitizer traces, reproducer code, issue discussions, and fix patches, is preserved verbatim. Deferring all abstraction to later stages allows agents to reason from the original evidence rather than from a lossy summary.

For each report, this stage collects:

- **Metadata:** issue metadata, including the title, labels, status, and creation date.
- **Discussion:** the complete issue body, all discussion comments, and linked text attachments such as logs and reproducer scripts.
- **References:** referenced pull requests, commits, and other fix links.
- **Fix:** details of each referenced fix, including its description or commit message and the patch for every changed file.

### *C. Abstraction and Clustering*

Normalized reports are detail-rich but unsuitable for comparison. Two reports with the same underlying failure mechanism in different modules share little vocabulary. TAXOSHOP therefore abstracts each report into a *workflow signature*, a short, implementation-neutral description of the bug. These workflow signatures are subsequently clustered for later stage analysis.

**Workflow Abstraction.** For each normalized report, an LLM produces an abstract signature consisting of an ordered sequence of steps and summary. These abstractions describe the causal sequence that leads to the failure and remove project-specific terminology, including function, API, and test names.

Formally, given normalized reports $\mathcal{R} = \{r_1, \ldots, r_n\}$ from a project $P$, TAXOSHOP maps each report to a workflow signature $s_i = \alpha(r_i) = (\omega_i, \sigma_i)$, where $\omega_i = \langle e_{i,1}, \ldots, e_{i,k} \rangle$ is a causally ordered sequence of $2 \leq k \leq 8$ steps ending in the failure, and $\sigma_i$ is a one-sentence summary. Signatures are instance-neutral. For the set $\mathcal{V}_P$ of identifiers in $P$ (e.g., function, API, and test names), $\mathrm{id}(s_i) \cap \mathcal{V}_P = \emptyset$.

**Hybrid Clustering.** TAXOSHOP clusters signatures in two phases. The first phase is embedding-based: TAXOSHOP embeds each signature and groups signatures by cosine similarity, producing an initial partition. The second phase is agentic refinement. An LLM examines the initial clusters and reassigns or consolidates them, merging clusters whose members share an underlying failure mechanism and separating members grouped only by surface similarity. The model is instructed to treat two reports as equivalent only when their failure workflow is the same, even if their symptoms, affected files, or implementation details differ, and it records a short justification for each change. TAXOSHOP validates every decision against a fixed output schema.

The two phases are complementary. Embedding similarity scales to the full corpus and is order-independent, but it is dominated by surface vocabulary: two reports that mention the same data structure appear close even when their mechanisms differ, and one mechanism described in different words appears distant. The refinement phase corrects this, because the model compares mechanisms directly rather than vocabulary. Deferring the final grouping to the model also avoids committing in advance to a cluster count or a similarity threshold as the definitive boundary, and each recorded justification makes the resulting clusters auditable.

### *D. Taxonomy Generation*

To characterize the clusters, the final stage turns each cluster into a *taxonomy entry* which precisely describes the cluster's recurrent bug pattern. The collection of entries forms the taxonomy. Entries serve two practical purposes. For researchers, it characterizes a recurring way in which the project fails. For auditors, it describes how to search for new instances.

Formally, clustering partitions $\mathcal{R}$ into clusters $\mathcal{C} = \{C_1, \ldots, C_m\}$. The taxonomy is the set of all entries, $\mathcal{T} = \{t_1, \ldots, t_m\}$. For each cluster, an LLM reads the full normalized document of every member report. From this information, the model determines which features are shared across the

cluster and which are specific to individual reports, and produces an entry

$$t_j = \gamma(C_j) = (\pi_j, \chi_j),$$

where $\pi_j$ names and briefly describes the pattern and $\chi_j = (pre, op, int, asm, fail)$ decomposes it into a contextual set:

- **Precondition** ($pre$): the state the code must be in for the bug to be possible.
- **Critical operation** ($op$): the action during which the bug is triggered.
- **Interference** ($int$): the event that invalidates the code's expectations during that operation.
- **Invalid assumption** ($asm$): the implicit belief that the interference violates.
- **Failure** ($fail$): the resulting misbehavior.

The set separates the bug's conditions from its individual instances and effects. The invalid assumption is its most important field: it names what the code gets wrong, which is both what a variant must share and, typically, what a fix must correct.

### E. Variant Analyzer

To demonstrate the practical utility of TAXOSHOP's taxonomy generation, we pair TAXOSHOP with a simple variant analyzer using a flag-verification loop [11]. We refer to the analyzer guided by TAXOSHOP taxonomies as TAXOSHOP-VA. Figure 2 shows its two phases.

**Location Flagging.** The agent selects a taxonomy entry and uses it to audit the interpreter source for code matching the pattern. For example, for the stale-state entries, it looks for native routines that retain a pointer, length, or borrowed reference across a call that can execute user code. Each candidate is recorded as a flag consisting of a file, a line range, and the taxonomy entry it instantiates. The agent keeps a running log of flagged locations and of modules it has audited and judged safe, so that later iterations do not re-audit the same code.

**Flag Validation.** For each flag, the agent writes a reproducer in the target language that attempts to trigger the suspected bug and runs it on a sanitizer-instrumented build of the interpreter. A flag is confirmed only if its reproducer triggers a sanitizer report whose fault location matches the flagged code. For each confirmed flag, the agent searches the project's issue tracker and commit history for existing reports of the same defect and discards duplicates. It then writes a report containing the root cause, the minimal reproducer, the full sanitizer output, and the taxonomy entry it instantiates. We review each report manually before disclosure.

## IV. EVALUATION

We implemented TAXOSHOP with approximately 1K lines of Python code. TAXOSHOP uses GPT Luna for the workflow description generation, clustering, and taxonomy generation. Claude Opus 4.8 is used for the agentic bug finding phase. As a bug oracle, generated programs are executed on

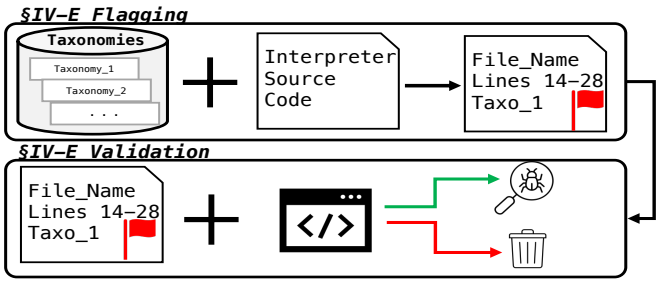



**Fig. 2: Overview of Bug Detection Pipeline**

an ASan/UBSan-instrumented build of each interpreter. We record sanitizer reports for triage and upstream reporting.

In this section, we evaluate TAXOSHOP to assess its ability to taxonomize and uncover new bugs in C-based interpreters and to understand the contribution of each design choice. Our evaluation answers the following research questions:

- **RQ1:** What is the effect of the clustering strategy?
- **RQ2:** What recurring bug patterns exist in CPython's history?
- **RQ3:** Can the mined taxonomy guide discovery of new bugs in CPython?
- **RQ4:** Do the patterns transfer to other interpreters (Ruby, PHP)?
- **RQ5:** How much does the taxonomy contribute compared with baselines?

### A. Experimental Setup

TAXOSHOP/TAXOSHOP-VA is deployed on a system with 8 cores and 32 GB of RAM running Ubuntu 22.04. Analyses are performed on the latest development versions of CPython, MRuby, and the official PHP interpreter to ensure all findings reflect current behaviors.

**Answering RQ1.** Clustering is crucial for grouping bugs into a usable taxonomy. To examine the effects of our hybrid clustering strategy, we compare against (1) embeddings-only clustering and (2) LLM-as-judge clustering.

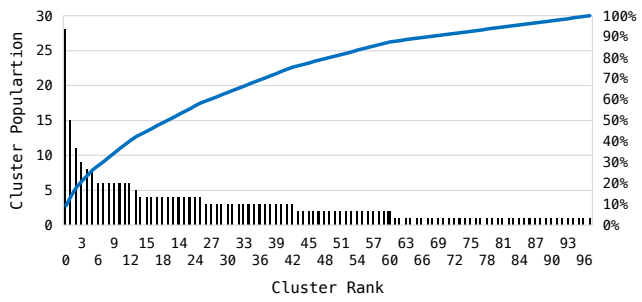
**Answering RQ2.** To identify the dominant bug patterns in CPython, we mine 361 memory-safety issues and their associated fixes, where available, from the CPython repository and use them as input to TAXOSHOP. We report the resulting cluster distribution and characterize the largest categories.

**Answering RQ3.** To examine whether the taxonomy can guide new bug discovery, we apply TAXOSHOP-VA to CPython with a $20 budget.

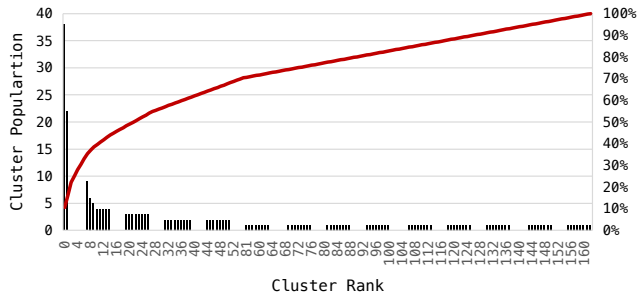
**Answering RQ4.** To assess whether the CPython taxonomy transfers to other C-based interpreters, we apply TAXOSHOP-VA to CRuby and PHP with a $20 budget per interpreter.

**Answering RQ5.** To isolate the contribution of the taxonomy, we compare TAXOSHOP-VA against the same variant analyzer given alternative guidance, under the same budget:

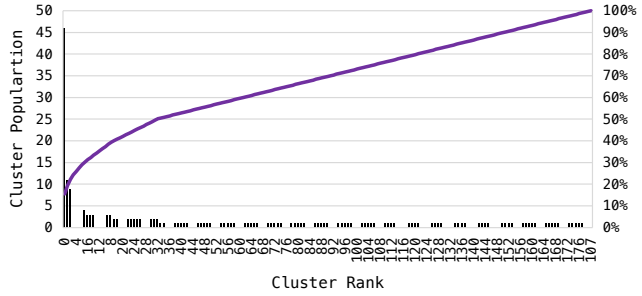
1) **Unguided:** a generic instruction to find memory-safety bugs.




**(a) Hybrid Clustering**




**(b) LLM-as-Judge Clustering**




**(c) Embeddings-based Clustering**

**Fig. 3: Pareto Distribution of CPython Clusters under three clustering approaches.**

2) **CWE-Guided:** descriptions of the CWE classes observed in the corpus.
3) **Instance-Guided:** individual CPython bug reports, without abstraction.

**Bug Oracle.** For memory and logic bug detection, we instrument each interpreter with `clang`'s AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan), both of which provide precise fault localization.

**Responsible Disclosure.** All bugs are responsibly disclosed to the appropriate development team following their respective guidelines. Each reported bug is unique as confirmed by development teams, requiring individualized patches.

### *B. Results*

*1) Clustering Effects:* TAXOSHOP clusters bug reports in two phases: an embedding pass that groups signatures by cosine similarity, followed by an agentic refinement pass that reassigns and consolidates clusters by failure mechanism (**??**). To justify this design, we compare it against each phase in isolation: a purely embedding-based method and an LLM-as-judge method that assigns each report to a cluster incrementally. Figure 4 shows the resulting cluster-size distributions.

The three methods differ sharply in how they treat the long tail. The embedding-based method leaves [81%] [verify] of its reports in singleton clusters, the most of any method, yet also produces the single largest cluster ([66] [verify] reports, [18%] [verify] of the corpus). This is the characteristic failure of surface similarity: reports that share vocabulary are merged into one dominant cluster regardless of mechanism, while reports describing the same mechanism in different terms are left unmatched. The LLM-as-judge method reduces singletons to [64%] [verify], but its result depends on the order in which reports are processed, since each assignment is made with knowledge of only the clusters formed before it.

The hybrid method produces the most balanced distribution. It reduces singletons to [37%] [verify] and roughly doubles the mean cluster size relative to either baseline ([3.6] [verify] versus [2.0] [verify] and [2.3] [verify]). At the same time, its largest cluster ([57] [verify] reports) is smaller than the embedding method's, indicating that the refinement pass both consolidates the tail and breaks up over-merged clusters. The effect is visible in Figure 4 as a curve that rises more gradually than the embedding method's and extends further before flattening into singletons. Concretely, [51%] [verify] of all reports fall in clusters of five or more members under the hybrid method, compared with [41%] [verify] and [39%] [verify] for the embedding and LLM-as-judge methods. A larger share of the corpus is therefore placed into recurring, populated categories, which are the categories a taxonomy entry can be synthesized from.

These results support the two-phase design. The embedding pass alone conflates vocabulary with mechanism; the refinement pass corrects both of its failure modes, moving reports out of an over-merged cluster and out of the singleton tail into mechanism-coherent groups. We use the hybrid clustering for the taxonomy evaluated in the remainder of this section.

**Answer to RQ1:** Hybrid clustering produces a more balanced distribution than both LLM-as-judge and embedding methods.

*2) CPython Taxonomy:* To characterize recurring patterns among memory-related bugs in CPython, we mine 361 memory-safety bug reports and their associated fixes from CPython's GitHub repository and process them with TAXOSHOP's taxonomy generation pipeline.

**Distribution.** TAXOSHOP groups the 361 reports into 164 clusters. As illustrated by Figure 4, the distribution is long-tailed. 107 clusters (65%) are singletons. 156 reports (43%) belong to clusters with five or more members with five clusters containing ten or more. Most memory bugs in CPython are therefore idiosyncratic, each arising from a specific wrong assumption in a specific location. A small number of mech-

| Rank | Cluster | Count | Description |
|---|---|---|---|
| 1 | C-0014 | 38 | Stale borrowed handles after invalidation. Code retains a non-owning pointer, reference, iterator, view, or cache entry while another operation can free, replace, resize, or tear down its target. Later use of that handle produces UAFs, invalid reads, or corruption. |
| 2 | C-0080 | 22 | Callback re-entrancy invalidates active native state. Native code calls user-defined behavior while holding raw pointers, cached sizes, iterator state, or container metadata. The callback mutates or destroys that state, and the native operation resumes with stale assumptions. |
| 3 | C-0137 | 20 | Re-entrant protocol/conversion invalidates a backing resource. A native operation crosses a user-code boundary for conversion, iteration, comparison, or callback dispatch while retaining a resource handle or pointer. Re-entrant code clears, closes, resizes, replaces, or releases it before subsequent native use. |
| 4 | C-0077 | 11 | Concurrent contexts race on process-wide state. Threads or isolated runtime contexts share lifecycle state, caches, registries, flags, or resources without complete synchronization. Initialization, publication, operation, or teardown overlaps and produces races or inconsistent state. |
| 5 | C-0091 | 10 | Error paths leak temporary owned state. A native operation allocates or acquires an intermediate value, then encounters invalid input, a failed callback, or another recoverable error. An incomplete cleanup path leaves the temporary allocation or reference live. |
| 6 | C-0081 | 9 | Concurrent unsynchronized access to mutable shared storage. Multiple threads access shared buffers, containers, iterators, fields, or external-library state without a complete common synchronization and lifetime protocol. This yields data races, inconsistent state, crashes, or stale accesses. |
| 7 | C-0056 | 9 | Concurrent mutation or release of shared state. An operation reads or uses a shared descriptor, pointer, dispatch value, buffer, or object field while another path can concurrently replace, close, reinitialize, or free it. Inadequate coordination causes races and sometimes UAFs. |
| 8 | C-0094 | 9 | Successful paths omit ownership release. A routine allocates auxiliary state during normal operation, initialization, registration, or teardown but neither transfers nor releases ownership on one completion path. Repeated use or shutdown then exposes leaks. |
| 9 | C-0074 | 6 | Lazy shared-state initialization races. Multiple threads observe an uninitialized cache or derived value and concurrently compute or publish it without a lock, atomic protocol, or once-only mechanism. Readers can observe partial state; ownership and reference accounting can race. |
| 10 | C-0034 | 5 | Process-global state survives interpreter lifecycles. Runtime or extension state retains interpreter-owned objects, metadata, caches, or allocator resources across interpreter shutdown and recreation. Later reuse causes leaks, stale-state access, invalid frees, or crashes. |

**TABLE I: Top bug clusters of CPython**

anisms, however, recur systemically (e.g., use of a borrowed reference after reentrant user code, races on shared process-wide state, and loss of ownership on error paths). The five largest clusters (`0014`, `0080`, `0137`, `0077`, and `0091`) together account for 101 reports, or 28% of the corpus.

**Dominant Patterns.** Table I lists the five largest clusters with their populations and summaries. For each cluster, TAXOSHOP produces a taxonomy entry that describes the pattern as a causal chain (precondition, critical operation, interference, invalid assumption, and failure) and includes a scope note that bounds the generalization.

*Stale state across invalidation.* Three of the five clusters (`wf-0014`, `wf-0080`, and `wf-0137`; 80 reports, 22% of the corpus) describe variants of a single hazard. Native code retains a non-owning handle, such as a borrowed reference, raw pointer, cached length, or iterator position, across an operation that can invalidate its target. It then resumes using that handle without reacquiring ownership or revalidating it. The clusters differ in where the invalidation comes from. `wf-0014` is the broadest: the target may be released by a reference-count decrement, garbage collection, container mutation, interpreter shutdown, or another thread. `wf-0080` and `wf-0137` both attribute the invalidation to user-defined code that the native operation invokes midway through hashing, comparison, conversion, iteration, formatting, or I/O. That code reenters the interpreter and mutates, resizes, detaches, or frees state the caller still depends on. The invalid assumption common to both is that a call into user code only observes state and never changes it. The two entries overlap substantially, and we discuss this overlap as a limitation of incremental clustering in **??**.

*Shared-state races.* `wf-0077` (11 reports) captures concurrent execution contexts, including threads, subinterpreters, free-threaded builds, and shutdown sequences, that access process-wide state without a complete synchronization protocol. Examples of such state include lazily initialized metadata, caches, flags, and registries. The invalid assumption is that the global interpreter lock, an isolation boundary, or a lifecycle phase serializes all accesses. Some of these races are benign in value and are detected only by race instrumentation.

*Error-path ownership loss.* `wf-0091` (10 reports) captures operations that acquire an owned intermediate value and then exit through an error path that fails to release it. The operation correctly reports a recoverable error but leaks the intermediate, and the leak surfaces under repeated calls or leak checking at exit. Nine reports establish this mechanism. The tenth has an unconfirmed cause, and the entry retains it only as a boundary case.

In all five clusters, the synthesis stage found that the member reports supported a broader pattern than the initial workflow hypothesis. For example, the evidence for `wf-0080` extends beyond cleanup of embedded containers, and the evidence for `wf-0091` extends beyond text parsing. Reading the full reports therefore corrects hypotheses that were anchored to the cluster's first member.

**Answer to RQ2:** CPython exhibits numerous recurrent bug patterns in its implementation.

**TABLE II: Bugs confirmed by development teams. Issue IDs are redacted for anonymized review.**

| # | System | Issue ID | Class | Description | Fix |
|---|---|---|---|---|---|
| 1∗ | Python | [redacted] | CWE-125 | Heap buffer overflow (OOB read of size 1) in _codecs_cn.c hz_decode on trailing ~ | +12 -1 |
| 2⋆ | Python | [redacted] | CWE-787 | Out-of-bounds write (SEGV) in mmapmodule.c mmap_ass_subscript via reentrant resize() | +67 -14 |
| 3† | Python | [redacted] | CWE-416 | Heap use-after-free (WRITE) in bufferedio.c _io__Buffered_read_impl via detach() during read() | |
| 4† | Python | [redacted] | CWE-416 | Heap use-after-free in bufferedio.c _bufferedreader_raw_read via detach() during read(n) | +48 -9 |
| 5× | Python | [redacted] | CWE-362 | Data race on process-global timezone state in timemodule.c get_zone via concurrent tzset() | +99 -4 |
| 6† | Python | [redacted] | CWE-476 | Member access within null pointer (segfault) in _sqlite/cursor.c _pysqlite_fetch_one_row | +32 -1 |
| 7‡ | Python | [redacted] | CWE-416 | Heap use-after-free (READ) in _lsprof.c ptrace_enter_call via reentrant enable() from external timer | +38 -1 |
| 8‡ | Python | [redacted] | CWE-416 | Heap use-after-free (READ) caused by a reentrant __eq__ freeing a sentinel while being compared | +44 -1 |
| 9‡ | Python | [redacted] | CWE-416 | Heap use-after-free (READ) caused by a reentrant __eq__ freeing a reference while still in use | +34 -1 |
| 10‡ | Python | [redacted] | CWE-416 | Heap use-after-free (READ) from a reentrant function prematurely freeing a sentinel | +44 -1 |
| 11† | Ruby | [redacted] | CWE-416 | Heap use-after-free (READ) in pack.c pack_unpack_internal via String#unpack mutating the receiver | +20 -0 |
| 12‡ | Ruby | [redacted] | CWE-125 | Heap buffer overflow (READ) in array.c flatten via reentrant to_ary shrinking the array | +137 -2 |
| 13⋆ | Ruby | [redacted] | CWE-125 | Heap buffer overflow (READ) in array.c rb_ary_zip via reentrant to_ary shrinking the receiver | +13 -1 |
| 14⋆ | Ruby | [redacted] | CWE-125 | Heap buffer overflow (READ) + negative-length Array corruption in array.c via to_int | |
| 15⋆ | Ruby | [redacted] | CWE-125 | Heap buffer overflow (READ) in array.c append_values_at_single via Range endpoint to_int | |
| 16† | Ruby | [redacted] | CWE-416 | Heap use-after-free (READ) in transcode.c transcode_restartable0 via encode mutating the source | +28 -3 |
| 17† | Ruby | [redacted] | CWE-416 | Heap use-after-free (READ) in io_buffer.c io_buffer_get_string via to_str resizing the buffer | |
| 18† | Ruby | [redacted] | CWE-787 | Heap out-of-bounds (WRITE) in pack.c pack_pack ('r'/'R') via to_int shrinking the buffer | +16 -1 |
| 19‡ | PHP | [redacted] | CWE-416 | Use-after-free (SEGV) in ext/sqlite3/sqlite3.c SQLite3Stmt::close() finalizing the running VDBE | |
| 20‡ | PHP | [redacted] | CWE-416 | Heap use-after-free (WRITE) in xpath.c php_xpath_eval via __construct() freeing context | |
| 21⋆ | PHP | [redacted] | CWE-416 | Heap use-after-free (READ) in xsltprocessor.c php_xsl_apply_stylesheet via freed running stylesheet | +96 -2 |
| 22‡ | PHP | [redacted] | CWE-416 | Use-after-free (SEGV) in php_zip.c php_zipobj_close via close() double-closing the archive | +121 -1 |
| 23‡ | PHP | [redacted] | CWE-476 | Use-of-reset VDBE (SEGV) in pdo_sqlite/sqlite_statement.c via closeCursor() from a UDF callback | |

† 0014, ∗ 0036, × 0077, ‡ 0080, ⋆ 0137.

*3) Bug Discovery:* To assess the practical utility of TAXOSHOP's generated taxonomies for variant analysis, we deploy TAXOSHOP-VA on CPython. The upper segment of Table II displays the previously unknown CPython bugs discovered by our system. Given a $20 USD budget, TAXOSHOP-VA discovers 10 new bugs in CPython.

The discovered bugs are spread across [eight] modules and span five weakness classes. Use-after-free (CWE-416) dominates with six bugs. The remaining four are an out-of-bounds read (CWE-125), an out-of-bounds write (CWE-787), a data race (CWE-362), and a null pointer dereference (CWE-476). At the time of writing, maintainers have confirmed all ten bugs and patched nine. With a $20 budget, this amounts to an average cost of $2 per confirmed bug.

**Pattern Coverage.** Every bug is an instance of a TAXOSHOP taxonomy entry, and their distribution mirrors the dominant patterns identified in RQ1. Eight of the ten belong to the stale-state family (`0014`, `0080`, and `0137`). In each, native code retains state across a call into user code or a reentrant interpreter operation, and that call invalidates the state before the native code uses it again. Bugs 3 and 4 illustrate the pattern. While a buffered read is in progress, the underlying raw stream's user-defined read method calls `detach()` on the buffered object; when control returns, the native read path accesses state that `detach()` released. Bug 7 has the same shape with a different trigger: a user-supplied profiler timer reenters the profiler by calling `enable()`, which frees state that the profiler's entry hook dereferences after the timer returns. The remaining two bugs instantiate other patterns. The timezone race (Bug 5) is an instance of `0077`. The codec overflow (Bug 1) is an instance of `0036`, a smaller cluster outside the five largest, which indicates that entries in the taxonomy's long tail also provide usable guidance.

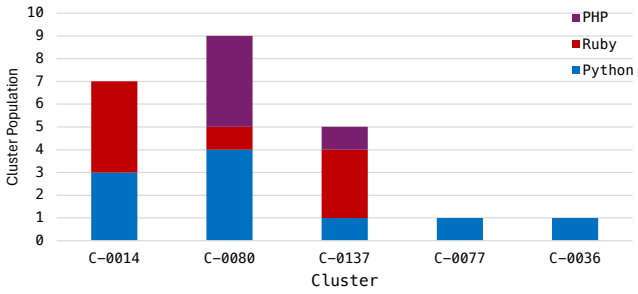



**Fig. 4: Cluster Distribution of New Bugs**

**Fix Characteristics.** The patches are small (median $+44/-1$ lines) and predominantly additive. Rather than restructuring the affected code, they add the reference, revalidation, or synchronization whose absence the corresponding taxonomy entry identifies as the invalid assumption. This suggests the entries capture not only where bugs occur but also why, in a form that points to the fix.

**Answer to RQ3:** TAXOSHOP-VA is able to discover 10 previously unknown bugs in CPython by searching for patterns identified by the taxonomy.

*4) Cross-Interpreter Transfer:* TAXOSHOP taxonomies describe bug patterns in implementation-neutral terms. To test whether this abstraction lets the taxonomies generalize beyond the project they were mined from, we apply TAXOSHOP-VA to CRuby and PHP using the taxonomies mined from CPython, without modification. We do not run TAXOSHOP on either interpreter's bug history. With a $20 budget per interpreter, TAXOSHOP-VA discovers 8 previously unknown bugs in CRuby and 5 in PHP, shown in the lower two segments

**TABLE III: Comparison of TAXOSHOP-VA with baselines on CPython, averaged over three campaigns. TP and FP are confirmed and false-positive findings per campaign; TTD is the time to the first confirmed bug.**

| Configuration | TP | FP | TTD (min) | Time (min) |
|---|---|---|---|---|
| Unguided | 1.0 | 0.0 | 19.3 | 53.7 |
| CWE-Guided | 3.3 | 0.0 | 11.0 | 61.7 |
| Instance-Guided | 1.7 | 3.3 | 8.7 | 60.0 |
| **TAXOSHOP-VA** | **9.0** | **0.0** | **6.0** | 64.0 |

of Table II. This amounts to an average cost of $2.50 and $4 per bug, respectively.

**Pattern Coverage.** All 13 bugs are instances of the stale-state family (`wf-0014`, `wf-0080`, and `wf-0137`). The concentration is even stronger than in CPython, where this family accounted for eight of ten bugs. The underlying mechanism is unchanged: native code retains a pointer, length, or resource handle across a call that can execute user code, and that code invalidates the retained state before the native code resumes. What differs across interpreters is the language feature through which user code reenters.

In CRuby, the reentry vector is the implicit conversion protocol. Core routines call `to_ary`, `to_int`, and `to_str` to coerce their arguments, and a user-defined conversion method can mutate the receiver or its buffer in the middle of the operation. Six of the eight CRuby bugs (Bugs 12–15, 17, and 18) follow this path. For example, in Bugs 12 and 13, `flatten` and `zip` cache an array's length, invoke `to_ary` on an element, and continue reading after the user-defined conversion has shrunk the array. This is the CRuby analogue of CPython's reentrant `__eq__` and `__index__` bugs.

In PHP, the bugs arise in extensions that wrap external C libraries (SQLite, libxml2, libxslt, and libzip). Here the reentry vector is an object lifecycle method, such as `close()`, `__construct()`, or `closeCursor()`, invoked from a callback while the library is still executing on the object's resource. The extension then resumes and uses a statement, context, stylesheet, or archive that the lifecycle method has already finalized.

In neither interpreter did TAXOSHOP-VA find instances of the shared-state race (`wf-0077`) or error-path ownership (`wf-0091`) patterns. One plausible explanation is that neither interpreter exposes shared mutable state to concurrent execution to the extent CPython does. CRuby serializes Ruby-level threads under a global VM lock, and PHP typically executes each request in isolation. We have not tested this explanation, and the absence of findings within a fixed budget does not establish that these patterns are absent.

**Answer to RQ4:** The CPython taxonomy is abstract enough such that it can meaningfully guide TAXOSHOP-VA to discover 13 bugs in the official Ruby and PHP interpreters.

*5) Comparison Study:* To measure the contribution of the TAXOSHOP taxonomy, we compare TAXOSHOP-VA against the same variant analyzer under three alternative forms of guidance: none (Unguided), CWE descriptions (CWE-Guided), and individual CPython bug reports (Instance-Guided). Table III reports the average number of true-positive (TP) and false-positive (FP) findings per campaign, the time to the first confirmed bug (TTD), and the total campaign time.

TAXOSHOP-VA outperforms all baselines, finding an average of 9.0 bugs per campaign, nearly three times as many as the strongest baseline, CWE-Guided (3.3). Instance-Guided and Unguided find 1.7 and 1.0 bugs, respectively. TAXOSHOP-VA also finds its first bug soonest, after 6.0 minutes on average, compared with 8.7 to 19.3 minutes for the baselines. Since all configurations run for a similar total time (54 to 64 minutes), these differences reflect the effectiveness of the guidance rather than differences in search effort.

Instance-Guided is the only configuration that produces false positives (3.3 per campaign). Given specific bug reports, more than other configurations, the analyzer tended to make reproducers of previously known bugs rather than new discoveries. It also finds fewer bugs than CWE-Guided, even though it receives more specific information. This supports the premise of TAXOSHOP: individual bug instances are too specific to generalize from, and CWE classes are too coarse to locate new bugs, whereas the taxonomy's categories describe mechanisms at a level that is both general and actionable.

**Answer to RQ5:** Over 3 campaigns, TAXOSHOP-VA is able to outperform other variant analysis techniques.

## V. CASE STUDIES

To illustrate how a single taxonomy entry guides discovery across interpreters, we examine three bugs found by TAXOSHOP-VA, one per language, all guided by the same CPython entry: `C-0080` (Figure 5). Figure 6 gives a proof of concept for each. Despite targeting three codebases with unrelated APIs, the three bugs share the same pattern: a native operation retains borrowed state across a call into user code, the user code reenters the interpreter and invalidates that state, and the operation then resumes using it.

### A. Case Study 1: CPython

The `_lsprof` profiler allows an external timer callback to run arbitrary Python code on each profiled event. That callback can recursively call `Profiler.enable()`, which is not guarded against reentrancy. The nested activation re-registers the monitoring callbacks and resets the profiler state while the outer event is still relyļng on its active `ProļilerContext` chain and freelist. When profiling is later disabled, cleanup frees a context that the outer event still references, and the next access is a heap use-after-free.

The bug follows the `C-0080` shape. The outer profiler operation retains borrowed internal state, its native context chain and freelist, across the timer callback. The callback reenters the same subsystem and mutates its lifecycle state. Execution then resumes and cleans up through the invalidated state.

```
# Callback re-entrancy invalidates borrowed state or
backing storage during a native operation

A low-level operation invokes user-defined behavior while
retaining borrowed references, raw pointers, cached
sizes, iterator state, or container metadata. The
callback re-enters and mutates, replaces, clears,
resizes, detaches, or destroys that state before the
operation resumes.

## Precondition
A native fast path holds unowned object references or
cached structural/storage state across a potentially
user-defined callback.

## Critical operation
The operation performs lookup, iteration, conversion,
comparison, formatting, serialization, packing, I/O,
insertion, or cleanup and invokes user code mid-operation.

## Interference
The callback re-enters the active object or its owner and
changes the relevant container, iterator, buffer, cache,
backing object, or object lifetime.

## Invalid assumption
The operation assumes the callback is observational, or
assumes previously acquired pointers, references,
lengths, indexes, and representation metadata remain
valid after the callback.

## Failure
Continuation through stale state causes use-after-free,
null dereference, out-of-bounds access, invalid
iteration, assertion failure, or process termination.

## Scope
The shared mechanism is callback-driven re-entrancy,
broader than embedded-container cleanup: callbacks arise
from hashing, equality, conversion, iteration,
formatting, I/O, serialization, and destruction. Some
reports invalidate object lifetime, while others
invalidate buffers, cached sizes, representation, or
iteration state without immediate deallocation; the
common boundary is continued use of unprotected state
after arbitrary user code.
```

**Fig. 5: Taxonomy entry for `C-0080`, as generated by TAXOSHOP.**

**(a) CPython: `_lsprof` (Bug 7)**

```python
import _lsprof
prof = _lsprof.Profiler()
def _timer():
    try:
        prof.enable()          # reenters the profiler
    except Exception:
        pass
    return 0
prof.__init__(timer=_timer)
prof.enable()
def foo():
    return 1
foo()                          # use-after-free
prof.disable()
```

**(b) CRuby: `Array#flatten` (Bug 12)**

```ruby
a = (1..2000).to_a
bad = Object.new
$a = a
def bad.to_ary; $a.clear; [1,2,3]; end   # shrinks the
    array
a << bad
a.flatten                                # heap buffer
    overflow
```

**(c) PHP: `ZipArchive::close()` (Bug 22)**

```php
<?php
$tmp = tempnam(sys_get_temp_dir(), 'zc');
$zip = new ZipArchive();
$zip->open($tmp, ZipArchive::CREATE | ZipArchive::
    OVERWRITE);
for ($i = 0; $i < 64; $i++) {
    $zip->addFromString("f$i.txt", str_repeat('x', 2000));
}
$GLOBALS['zip'] = $zip;
$zip->registerProgressCallback(0.0, function ($r) {
    static $done = false;
    if (!$done) { $done = true; $GLOBALS['zip']->close();
     } // reenters close()
});
$zip->close();                   // use-after-free
```

**Fig. 6: Proofs of concept for bugs discovered by TAXOSHOP-VA in each interpreter. All three are instances of `C-0080` (Figure 5): a user-defined callback reenters the interpreter and invalidates state that native code continues to use.**

### *B. Case Study 2: CRuby*

`Array#flatten` scans the receiver for an element convertible to an array, invoking each candidate's user-defined `to_ary` during the scan. In the PoC, the malicious element follows 2,000 ordinary ones. Its `to_ary` clears the receiver and shrinks the array's heap buffer. However, `flatten` has cached the pre-callback index (`i = 2000`) and resumes by copying that many elements from the now-smaller buffer into a result array sized from the post-clear length. The stale bound makes the copy read past the end of the shrunken buffer and write past the end of the under-allocated result.

This is again `C-0080`, but the retained state is not a pointer. It is the cached length and index that determine the copy size. The `to_ary` callback invalidates that cached bound by shrinking and reallocating the receiver's storage, and the native copy proceeds under the stale bound.

### *C. Case Study 3: PHP*

`ZipArchive::close()` commits an archive through libzip's `zip_close()`, during which libzip invokes the registered progress callback in userland. In the PoC, that callback recursively calls `ZipArchive::close()` on the same object. The wrapper has not yet cleared its native archive pointer, so the nested call runs `zip_close()` on the archive already being closed. The inner close frees the native `zip_t`, and when the outer `zip_close()` resumes it operates on freed memory and crashes.

The pattern matches `C-0080`. The outer close retains a live native archive handle across a user-controlled callback. The callback reenters the same API and destroys that handle, and the resuming outer close dereferences the invalidated resource.

## VI. THREATS TO VALIDITY

**Internal Validity.** TAXOSHOP relies on LLMs, whose outputs are nondeterministic. A different run may produce different signatures, cluster assignments, and entries. Clustering is also incremental: each report is assigned with knowledge of only

the clusters formed before it, so the resulting taxonomy depends on processing order. We process reports in a fixed order (ascending issue number), but we have not measured how much the taxonomy varies across orders. One visible consequence is overlap between categories. For example, `wf-0080` and `wf-0137` describe closely related mechanisms, and several bugs in Table II plausibly instantiate either. Our counts of bugs per category should therefore be read as approximate. The same nondeterminism affects the variant analyzer. We perform a single run per configuration, so differences between configurations in RQ4 may partly reflect run-to-run variance.

The LLMs we use were likely trained on CPython's source code and public issue tracker, and some taxonomy entries may reflect the model's prior knowledge rather than the mined evidence alone. This threat does not apply to the discovered bugs themselves, which were previously unreported. To limit unsupported generalization, TAXOSHOP grounds each entry in the full text of its member reports and rejects entries whose evidence does not cover every member.

**External Validity.** We construct a single taxonomy, from 363 memory-safety reports in one project, and evaluate its transfer to two other interpreters. All three are mature interpreters written in C, with similar object models and memory-management strategies. The taxonomy's transfer may not extend to interpreters written in other languages, to other categories of software, or to bug classes other than memory safety. Our selection of reports [describe criterion] may also bias the taxonomy toward bugs that were reported with sanitizer evidence. Finally, our results are obtained with specific models (GPT-5.6 Luna and Claude Opus 4.8) and a fixed budget; other models or budgets may yield different taxonomies and different numbers of discovered bugs.

## VII. Related Work

**Variant Analysis.** Clone-based [**?**], [7], patch-based [**?**], and directed-fuzzing [**?**] approaches all start from a single known bug and search for code that resembles it. IRIS [**?**] uses an LLM to infer specifications for a static analyzer, and Big Sleep [**?**] uses an LLM agent to search for variants of a recently fixed bug. Both, however, still seed the search with an individual bug or a generic weakness class. TAXOSHOP instead seeds variant analysis with patterns abstracted from many bugs, and from the project in which they were found.

**Bug Taxonomies and Empirical Studies.** Empirical bug studies have characterized failures in open-source software [**?**], [**?**], concurrency bugs [**?**], compiler bugs [**?**], deep learning systems [**?**], and memory-safety bugs in Rust [**?**]. These studies typically derive their taxonomies through manual qualitative coding, which limits their scale. Their categories are also designed to describe bugs, not to guide the search for new ones. General-purpose classifications such as CWE are broad enough to apply across projects but, as discussed in **??**, too coarse to locate new instances. TAXOSHOP automates taxonomy construction and structures each entry around the invalid assumption that a variant must share.

**LLMs for Mining Software Artifacts.** LLMs have been applied to bug reports to reproduce bugs [**?**], and to large text corpora to induce taxonomies and label documents [**?**]. TAXOSHOP applies LLMs to taxonomy induction over bug reports. It differs in grounding each category in the full evidence of its member reports, including sanitizer output and fix patches, and in targeting categories that are actionable for variant analysis rather than descriptive labels.

**Interpreter Testing.** Fuzzers for language implementations generate programs that exercise interpreter and compiler internals. Examples include Fuzzilli [**?**] and DIE [**?**] for JavaScript engines, PolyGlot [**?**] for multiple languages, and LLM-based fuzzers such as Fuzz4All [**?**] and WhiteFox [**?**]. These tools explore the input space broadly and do not draw on a project's bug history. TAXOSHOP is complementary: its taxonomy identifies which mechanisms recur and where to look, and could serve as guidance for a fuzzer as well as for the agent-based analyzer we use.

## VIII. Conclusion

We presented TAXOSHOP, an approach that uses LLMs to mine a project's bug history into a taxonomy of recurring bug patterns. TAXOSHOP normalizes bug reports while preserving their evidence verbatim, abstracts each report into a project-neutral workflow, clusters reports that share a failure mechanism, and describes each cluster as a causal chain grounded in the full evidence of its members. Applied to 363 memory-safety reports from CPython, TAXOSHOP produces a taxonomy of 164 categories. Most CPython memory bugs are idiosyncratic, but five systemic patterns account for 28% of all reports. The most prominent of these is native code using state that reentrant user code has invalidated.

To demonstrate the taxonomy's practical utility, we used it to guide a simple variant analyzer, which discovered 23 previously unknown bugs: 10 in CPython, 8 in CRuby, and 5 in PHP. Although the taxonomy was mined solely from CPython, it transferred to both other interpreters, where the same patterns appeared through each language's own reentry mechanisms. [Taxonomy guidance outperformed unguided, CWE-guided, and instance-guided baselines.] These results suggest that a project's bug history encodes knowledge that, once abstracted from its implementation details, applies beyond the project itself.

In future work, we plan to apply TAXOSHOP to other projects and bug classes, to combine taxonomies mined from multiple projects, and to use taxonomy entries as guidance for other bug-finding techniques such as fuzzing. Our dataset, taxonomy, and tooling are available at [**?**].

## References


[1] W3Techs. Usage statistics and market share of php for websites. https://w3techs.com/technologies/details/pl-php. Accessed: August 20, 2026.

[2] TIOBE Software BV. TIOBE Index for September 2026. https://www.tiobe.com/tiobe-index/, September 2026. Accessed: 2026-09-28.

[3] Yifan Wu, Xiaochuan Yu, and Zhiyun Qian. Melting the flesh of PHP's memory hardening. In *35th USENIX Security Symposium (USENIX Security 26)*, pages 2803–2821, Baltimore, MD, August 2026. USENIX Association.

[4] Han Zheng, Flavio Toffalini, Qiang Liu, and Mathias Payer. Squeezing juicy variant bugs out of modern browsers. In *20th USENIX WOOT Conference on Offensive Technologies (WOOT 26)*, pages 131–151, Baltimore, MD, August 2026. USENIX Association.

[5] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. Generalized vulnerability extrapolation using abstract syntax trees. In *Proceedings of the 28th Annual Computer Security Applications Conference*, ACSAC '12, pages 359–368. ACM, 2012.

[6] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, H. Qi, and Jie Hu. VulPecker: An automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications*, ACSAC '16, pages 201–213. ACM, 2016.

[7] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. VUDDY: A scalable approach for vulnerable code clone discovery. In *2017 IEEE Symposium on Security and Privacy*, pages 595–614. IEEE, 2017.

[8] Shouguo Yang, Zhengzi Xu, Yang Xiao, Zhe Lang, Wei Tang, Yang Liu, Zhiqiang Shi, Hong Li, and Limin Sun. Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis. *ACM Transactions on Software Engineering and Methodology*, 32(6):1–29, 2023.

[9] Claire Wang, Ziyang Li, Saikat Dutta, and Mayur Naik. QLCoder: A query synthesizer for static analysis of security vulnerabilities. In *International Conference on Learning Representations*, 2026.

[10] Arash Ale Ebrahim, Ali Abbasi, and Nils Ole Tippenhauer. DNA: Automated vulnerability variant detection through ai-synthesized queries. In *Proceedings of the 19th European Workshop on Systems Security*, EuroSec '26, pages 74–80. ACM, 2026.

[11] Junyoung Park and Insu Yun. Agentic fuzzing: Opportunities and challenges, 2026.